

Criptografia - Trabalho 1

Caio Henrique Ramos Rufino - GRR20224386

1. Introdução

As cifras desenvolvidas neste trabalho foram inspiradas na sequência de Fibonacci. Aqui serão apresentados os detalhes de como cada técnica funciona, além de detalhes da implementação e desempenho quando comparado com o AES.

2. Geração da Chave

O primeiro passo do algoritmo é derivar duas chaves numéricas, k_1 e k_2 , a partir de uma chave de texto fornecida pelo usuário. Primeiramente, a chave de texto é dividida em dois grupos: caracteres em posições pares (0, 2, 4...) e caracteres em posições ímpares (1, 3, 5...). k_1 é calculado como a soma dos valores decimais UTF-8 dos caracteres nas posições pares, e k_2 é a soma dos valores para os caracteres nas posições ímpares.

Exemplo para a chave "chave":

- Posições pares ('c', 'a', 'e'):
 - $k_1 = \text{ord}('c') + \text{ord}('a') + \text{ord}('e') = 99 + 97 + 101 = 297$
- Posições ímpares ('h', 'v'):
 - $k_2 = \text{ord}('h') + \text{ord}('v') = 104 + 118 = 222$

Após o cálculo, os valores são ordenados para garantir que k_1 seja sempre menor ou igual a k_2 . No exemplo acima, os valores seriam trocados, resultando em $k_1 = 222$ e $k_2 = 297$.

3. Substituição

Para a cifra de substituição juntaram-se as ideias de uma cifra de César e a sequência de fibonacci. Basicamente, cada caractere do texto original será deslocado de acordo com o número da sequência de fibonacci adaptada para aquela posição. A tabela a seguir ilustra isso:

1	2	3	5	8
T	E	S	T	E

Nesse caso, o primeiro caracter, “T”, seria deslocado uma posição para frente, resultando em “U”, o segundo caracter, “E” duas posições para a frente, resultando em “G”, e assim por diante.

Para calcular o deslocamento temos as variáveis $f1$ e $f2$, que são inicializadas com

$f1 = k1 \% \text{ modulo}$

$f2 = k2 \% \text{ modulo}$

O uso do módulo é necessário para evitar que a sequência atinja valores muito altos, o que afeta de forma drástica o desempenho, então, foi definido um patamar arbitrário chamado modulo de 16834 (o patamar deve ser definido levando em conta diversas razões, mas, principalmente, o “alfabeto” sendo utilizado).

A cada iteração, os valores de $f1$ e $f2$ são atualizados assim (novamente, o modulo é necessário para evitar que os valores aumentem absurdamente):

$f1 = f2$

$f2 = (f1 + f2) \% \text{ modulo}$

Durante a i -ésima iteração, o caractere na posição i recebe o deslocamento igual ao valor de “ $f1 \% \text{ modulo}$ ” naquela iteração.

Exemplo para $k1 = 1$ e $k2 = 2$ e modulo = 10

1	2	3	5	8	3	1	4	5
T	E	S	T	A	N	D	O	

Depois do 8 temos 3, pois $5 + 8 = 13$, $13 \% 10 = 3$. O 1 vem de $8 + 3 = 11$, $11 \% 10 = 1$...

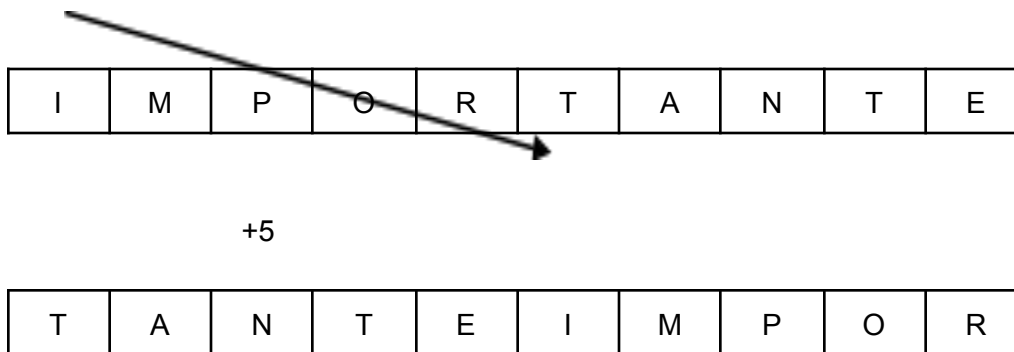
4. Transposição

A cifra de transposição segue a ideia de rotacionar blocos. Os blocos foram definidos como tendo o tamanho 10, e o deslocamento, ou seja, o quanto o bloco será rotacionado é calculado basicamente da mesma forma como a substituição é calculada, com a diferença que o módulo utilizado é o próprio tamanho do bloco. O tamanho fixo de bloco serve para melhorar o desempenho, afinal, quanto menor o tamanho do bloco mais operações serão necessárias e menor será o desempenho.

Um exemplo da transposição está a seguir. A primeira coluna representa o valor da rotação para aquele bloco.

bloco	rotação	Bloco de texto de tamanho 10									
0	1	P	A	C	I	Ê	N	C	I	A	_
1	2	É	_	U	M	A	_	V	I	R	T
2	3	U	D	E	_	M	U	I	T	O	_
3	5	I	M	P	O	R	T	A	N	T	E

Exemplo do bloco 3 sendo rotacionado em 5:



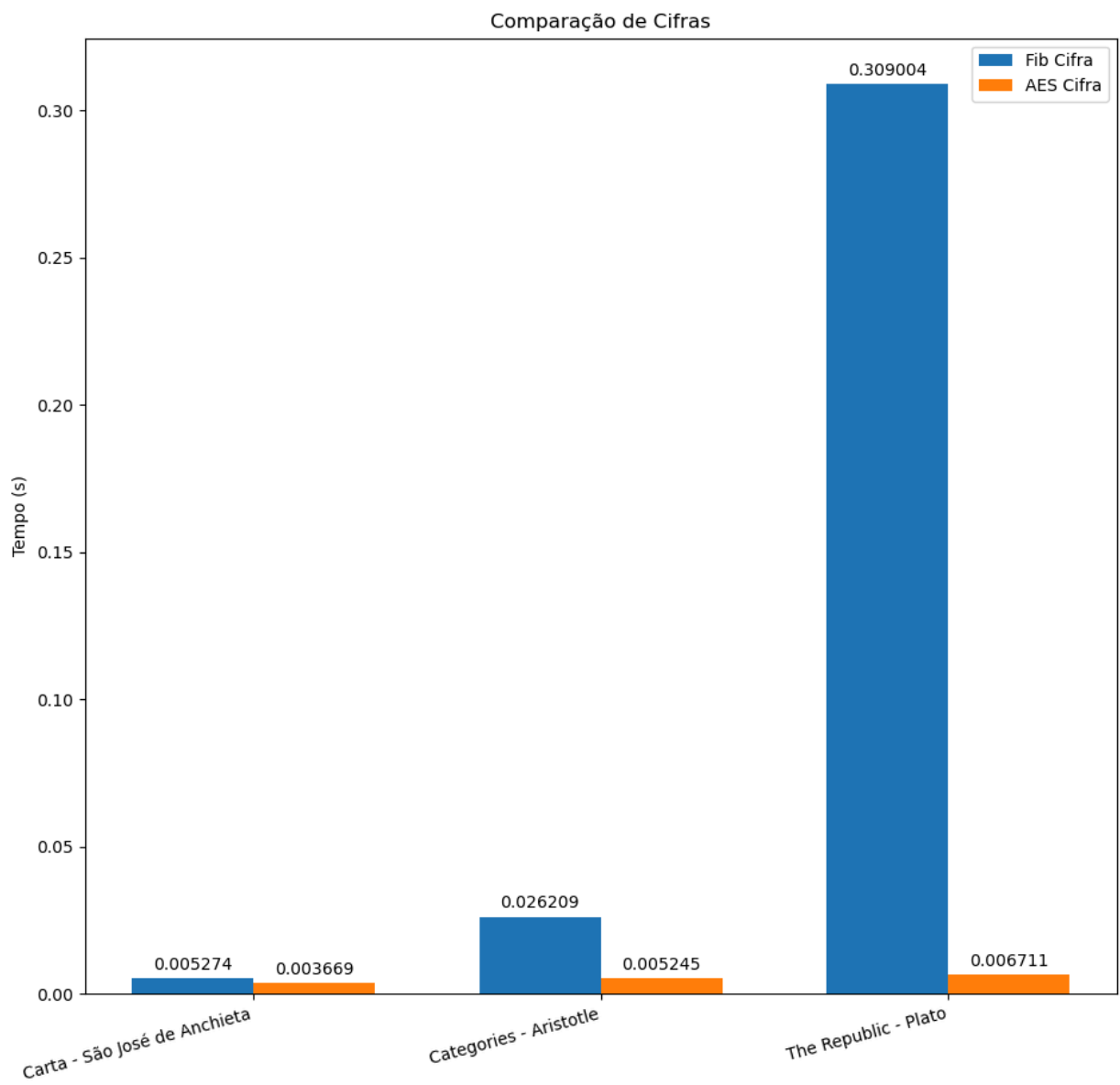
5. Processo de Cifragem e Decifragem

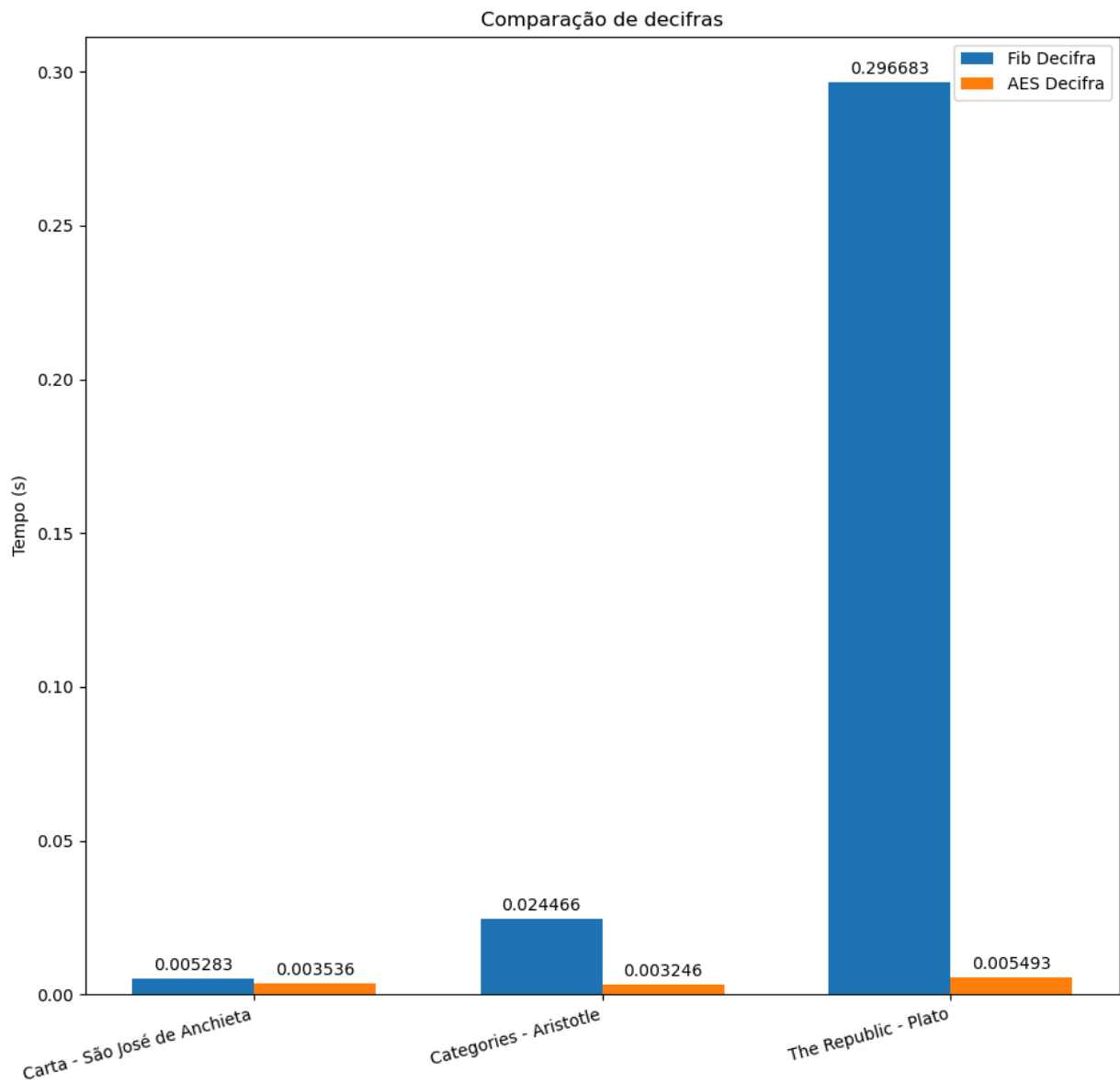
O algoritmo final combina as duas técnicas apresentadas acima. A ordem é a seguinte:

- Cifragem:
 1. Realiza-se a cifra de substituição sobre o texto original.
 2. O texto substituído é transposto pela cifra de transposição.
- Decifragem (ordem inversa e operações inversas da cifragem):
 1. De-transposição: O texto cifrado é dividido nos mesmos blocos de 10. A mesma sequência de Fibonacci (com módulo 10) é gerada para determinar os valores de rotação. Para reverter a rotação para a direita, aplica-se uma rotação para a esquerda (deslocamento circular à esquerda) em cada bloco.
 2. De-substituição: A mesma sequência de Fibonacci da cifragem (com MODULO 16384) é gerada. Para reverter o deslocamento, o valor de f1 de cada etapa é subtraído do valor decimal do caractere cifrado, em vez de somado.

6. Comparação com o AES

Todos os arquivos foram criptografados com a chave “chave”. Os arquivos estão em ordem de tamanho: 23K, 101K e 1.2M respectivamente.





7. Implementação e Uso

O algoritmo foi implementado em Python.

Uso:

```
python fibonacci_cipher.py <arquivo> <chave> [--decrypt] [--time]
```

- <arquivo>: O caminho para o arquivo de entrada.

- <chave>: A chave de texto para gerar k1 e k2.
- --decrypt: Flag opcional para ativar o modo de decifragem.
- --time: Flag opcional para medir e exibir o tempo de execução da operação.

O programa lê o conteúdo do arquivo especificado, realiza a cifragem ou decifragem e salva o resultado em um novo arquivo com o sufixo “-cifrado.txt” ou “-decifrado.txt”.